

Power Platform Skill Suite

ชุด Claude Skills 5 ตัว สำหรับสร้าง Power Apps, SharePoint list และ Power Automate flow ตั้งแต่รับ requirement จนถึงคู่มือผู้ใช้

เอกสารนี้ตอบ 4 เรื่อง

- 1 มันคืออะไร และแก้ปัญหาอะไร
- 2 ติดตั้งและใช้งานยังไง
- 3 มี skill อะไรบ้าง แต่ละตัวทำอะไร
- 4 ทิมแก้ไขและพัฒนามันต่อเองยังไง

ข้อสำคัญ

ตั้งแต่รอบนี้เป็นต้นไป ทิมเป็นเจ้าของและดูแล **skill** เหล่านี้เองทั้งหมด ไม่มีคนกลางคอยรวบรวม และปล่อยเวอร์ชันใหม่ให้อีกแล้ว

ส่วนที่ 7 อธิบายวิธีแก้ไขและพัฒนาต่อด้วยตัวเอง โดยไม่ต้องใช้ GitHub ไม่ต้องลงโปรแกรม และไม่ต้องเขียนโค้ด ใช้แค่การแชทกับ Claude ตามปกติ

– สารบัญ

1	ภาพรวม อันนี้คืออะไร · แก้ปัญหาอะไร · กฎมาจากไหน	3
2	ติดตั้ง 3 ขั้นตอน ทำครั้งเดียว	4
3	ลำดับการทำงานของโปรเจกต์ แชทไหนทำอะไร · จุดที่ต้องคุยกับ user	5
4	skill ทั้ง 5 ตัว แต่ละตัวรับอะไร ให้อะไร แก้ปัญหาอะไร	6–7
5	กติกการใช้ให้ประหยัด token 4 ข้อ สำหรับ Pro plan	8
6	ข้อจำกัดของ validator สิ่งที่ตรวจได้ และสิ่งที่ตรวจไม่ได้	9
7	การดูแลและพัฒนา skill ต่อ 7.1 บทบาทและที่เก็บ · 7.2 เก็บบันทึก · 7.3 แชทปรับปรุง · 7.4 กฎ 7 ข้อ	10–14
8	คำถามที่พบบ่อย	15
9	Checklist หน้าเดียว พิมพ์แปะไว้ได้	16

– ถ้ามีเวลาแค่ 2 นาที

- ติดตั้ง skill ทั้ง 5 อัน (ส่วนที่ 2 · หน้า 4)
- เปิดแชทใหม่ พิมพ์สั้น ๆ ว่าจะทำอะไร ไม่ต้องเรียกชื่อ skill
- ก่อนปิดแชท วาง prompt เก็บบันทึก (ส่วนที่ 7.2 · หน้า 11)

ในโฟลเดอร์ที่ได้รับ มีอะไรบ้าง

[skills/](#) – 5 ไฟล์ .zip อัปโหลดเข้า claude.ai
[prompts/team-quick-feedback.md](#) – วางท้ายแชททำงาน
[prompts/improve-the-skill.md](#) – ใช้ตอนปรับปรุง skill
[prompts/project-style-override.md](#) – โปรเจกต์ที่ใช้ดีไซน์คนละชุด
[CHANGELOG.md](#) – ประวัติการแก้ไขทุกเวอร์ชัน
[READ-ME-FIRST.txt](#) – 3 บรรทัดสำหรับคนที่เพิ่งเปิดไฟล์

1 ภาพรวม

1.1 อันนี้คืออะไร

Claude Skill คือชุดกฎและตัวอย่างที่อัปโหลดเข้า claude.ai ครั้งเดียว แล้ว Claude จะใช้กฎเหล่านั้น ทุกครั้งที่เจองานประเภทนั้น โดยที่เราไม่ต้องพิมพ์อธิบายซ้ำ

ชุดนี้มี 5 skill ครอบคลุมงาน Power Platform ตั้งแต่รับ requirement จนถึงคู่มือผู้ใช้ กฎข้างในวัดมาจากไฟล์ที่ใช้งานได้จริง ไม่ได้มาจากเอกสารหรือการคาดเดา

1.2 แก้ปัญหาอะไร

ปัญหาเดิม	ชุดนี้ทำอะไรกับมัน
ใช้เวลาขึ้น Power Apps เยอะกว่าจะเลื่อนแต่ละกล่องได้แต่ละอัน	ออกแบบและตกลงกับ user บนกระดาษก่อน (ส่วนที่ 3) แล้วค่อยสร้าง จากนั้นได้ YAML ที่ paste เข้า Studio ได้ทั้งหน้าในครั้งเดียว
Claude generate Power Apps ไม่เก่ง ต้อง prompt เยอะ ติงงานกลับไปมานานกว่าจะได้ YAML ที่ไม่ error	กฎที่ต้องเคยพิมพ์บอกมันทุกครั้ง ถูกย้ายไปอยู่ใน skill แล้ว และมี validator ตรวจสอบให้ก่อนส่งงานทุกครั้ง

ทำไม Claude ถึง generate Power Apps ไม่เก่งตั้งแต่แรก

ไม่ใช่เพราะเขียนโค้ดไม่เป็น แต่เพราะไม่รู้กฎเฉพาะของ Power Apps ที่คนทำงานต้องเจอเองถึงจะรู้ เช่น control type ไหนมีอยู่จริง เวอร์ชันต้องเป็นเลขอะไร property เรียงยังไง ค่าแบบไหนทำให้ YAML พัง เมื่อไม่รู้ มันจะเดา และคำตอบที่เดามาจะดูถูกต้องเสมอ จึงไม่มีใครรู้ตัวจนกด paste แล้ว error

1.3 กฎในชุดนี้มาจากไหน

แหล่ง	ปริมาณ
หน้าจอ Power Apps ที่ compile ผ่าน Studio	17 หน้าจอ · 2,830 controls · 2,847 property blocks
SharePoint list ที่ใช้งานจริงใน production	10 lists · 131 columns
Power Automate package ที่ export จาก tenant จริง	1 ไฟล์
บันทึกจากการใช้งานจริงหลังส่งมอบ	1 โปรเจกต์ (ChopChop) · 191 controls

กฎข้อไหนมีตัวเลขกำกับ แปลว่าวัดมาแล้ว ไม่ใช่ความเห็น และเมื่อเอกสาร convention ขัดกับไฟล์ที่ใช้งานได้จริง ไฟล์ชนะเสมอ — มี 7 ค่าที่เอกสารเดิมกำหนดไว้ แต่ไม่ปรากฏเลยสักครั้ง ใน 17 หน้าจอ

2 ติดตั้ง

ทำครั้งเดียว ใช้เวลาประมาณ 2 นาที ไม่ต้องลงโปรแกรมอะไรเพิ่ม

2.1 ขั้นตอน

- 1 เปิด [claude.ai](#) แล้วไปที่ **Settings → Capabilities → Skills**
- 2 กด **Upload skill**
- 3 เลือกไฟล์ `.zip` จากโพลเดอร์ `skills/` ที่ละอัน จนครบ 5 อัน
- 4 เสร็จแล้ว ไม่ต้องตั้งค่าอะไรเพิ่ม

2.2 ข้อควรรู้เรื่องเวอร์ชัน

เรื่อง	รายละเอียด
Skill ไม่ sync ระหว่างคน	เป็นของแต่ละ account ทุกคนต้องอัปโหลดเอง
อัปเดตแล้วต้องโหลดใหม่	เวอร์ชันใหม่ที่วางในโพลเดอร์กลาง จะไม่ถึงใครจนกว่าแต่ละคนจะอัปโหลดตัวเอง
วิธีอัปเดต	อัปโหลดไฟล์ใหม่ทับชื่อเดิม ไม่ต้องลบของเก่าก่อน
ดูว่าใช้เวอร์ชันไหนอยู่	ถามในแชทว่า "which version of the skill are you using?"

2.3 ใช้งานครั้งแรก

เปิดแชทใหม่ แล้วพิมพ์งานที่จะทำตรง ๆ **ไม่ต้องเรียกชื่อ skill** ไม่ต้องอธิบายกฎ Power Apps ไม่ต้องเปะ convention

พิมพ์แบบนี้	skill ที่จะทำงาน
"ช่วยวางระบบขออนุมัติเปิดบัญชีให้หน่อย"	planning-powerplatform-solutions
"ทำหน้านี้เป็น Power App ให้หน่อย"	generating-powerapps-yaml
"ทำไม gallery ขึ้นแค่ 500 แถว"	building-sharepoint-lists
"flow import ไม่ได้ ขึ้น connection ผิดบาท"	building-powerautomate-flows
"เขียนคู่มือให้หน่อย"	writing-app-manuals

ถ้าพิมพ์ไปแล้ว skill ไม่ทำงาน

ให้บันทึกไว้ตามส่วนที่ 7.2 นั้นแปลว่าคำอธิบายของ skill เขียนไม่ครอบคลุมวิธีที่คนถามจริง ซึ่งแก้ได้ในรอบปรับปรุง ไม่ใช่เรื่องที่ต้องพิมพ์ให้ถูกใจมัน

วิธีแก้เฉพาะหน้า: พิมพ์ชื่อ skill ไปตรง ๆ เช่น "use generating-powerapps-yaml"

3 ลำดับการทำงานของโปรเจกต์

หนึ่ง stage ต่อหนึ่งแชท เรียงตามลำดับนี้

ได้ requirement มา



แชทที่ 1 • planning-powerplatform-solutions

สัมภาษณ์ requirement แล้วได้ `solution-spec.yaml` + mockup HTML + เอกสารภาพรวมระบบสำหรับพิมพ์เป็น PDF



จุดตัดสินใจ

เอา PDF และ mockup ไปคุยกับ user ที่ขั้นตอนนี้ ก่อนสร้างอะไรทั้งสิ้น

แก้แบบตอนเป็นกระดาษใช้เวลา 5 นาที แก้ตอนขึ้น Power Apps แล้วใช้เวลาครึ่งวัน นี่คือจุดที่ประหยัดเวลาได้มากที่สุดของทั้งกระบวนการ



แชทที่ 2

building-sharepoint-lists
ไฟล์ .xlsx สำหรับ copy ลง
SharePoint

แชทที่ 3

generating-powerapps-yaml
ไฟล์ .pa.yaml สำหรับ paste เข้า
Studio

แชทที่ 4

building-powerautomate-flows
ไฟล์ .zip สำหรับ import



แชทที่ 5 • writing-app-manuals

คู่มือ admin และคู่มือผู้ใช้ เป็นไฟล์ Word

3.1 ไฟล์ที่เดินทางข้ามแชท

`solution-spec.yaml` คือไฟล์เดียวที่ต้องพกไปด้วยทุกแชท แชทที่ 1 เขียนมันขึ้นมา อีกสี่แชทอ่านมัน และแต่ละแชทอ่านเฉพาะส่วนของตัวเอง

แนบเป็นไฟล์ อย่า **paste** เนื้อหาลงในช่องแชท แล้วพิมพ์สั้น ๆ ว่าจะทำอะไร เท่านั้นไม่ต้องเล่าระบบใหม่ทั้งหมดอีกรอบ

4 skill ทั้ง 5 ตัว

4.1 planning-powerplatform-solutions

พูดว่า	"ช่วยวางระบบขออนุมัติเปิดบัญชีให้หน่อย" แล้วแนบ flowchart, ER diagram หรือเอกสาร process ที่มีอยู่
ได้อะไร	<code>solution-spec.yaml</code> · mockup HTML · เอกสารภาพรวมระบบ
แก้ปัญหา	การเริ่มขึ้นแอปก่อนตกลงแบบกับ user
ต้องรู้	ทำตัวนี้ก่อนเสมอ อีกสี่ตัวอ่าน spec ที่มันเขียน · มันสร้าง glossary ก่อนเรื่องอื่นทั้งหมด เพราะของสิ่งเดียวที่ถูกเรียกสองชื่อ จะกลายเป็นสอง list สองหน้าจอ แล้วต้อง migrate ที่หลัง · มันหยุดสัมภาษณ์เองเมื่อ spec ผ่าน validator แล้ว

4.2 generating-powerapps-yaml

พูดว่า	"ทำหน้านี้เป็น Power App ให้หน่อย" แล้วแนบ mockup, screenshot, HTML หรือ spec
ได้อะไร	ไฟล์ <code>.pa.yaml</code> สำหรับ paste เข้า Power Apps Studio
แก้ปัญหา	ปัญหาข้อ 2 ในตารางหน้า 3 โดยตรง
ต้องรู้	เป็นตัวที่ลงแรงมากที่สุดและผ่านการใช้งานจริงแล้ว · รู้ว่ามี control อยู่ 9 ชนิด เวอร์ชันเลขอะไร property เรียงอย่างไร · รัน validator ให้ก่อนส่งงานทุกครั้ง · อ่านส่วนที่ 6 ก่อนเชื่อผลของ validator

4.3 building-sharepoint-lists

พูดว่า	"สร้าง list ตาม spec นี้ให้หน่อย" แล้วแนบ <code>solution-spec.yaml</code>
ได้อะไร	ไฟล์ <code>.xlsx</code> สำหรับ copy ลง SharePoint และสคริปต์สร้าง column
แก้ปัญหา	การพิมพ์ column ที่ละอันในเบราว์เซอร์ และกับดัก delegation
ต้องรู้	มันจะไม่ใช้ Lookup column ให้ เพราะ Lookup ไม่ delegate แล้ว gallery จะตัดข้อมูลที่ 500 แถวโดยไม่มี error ใด ๆ ขึ้น · 131 columns ที่ใช้งานจริงไม่มี Lookup สักอัน และนั่นเป็นความตั้งใจ

4.4 building-powerautomate-flows

พูดว่า	"ทำ flow ส่งเมลตอนมีคนกดส่งคำขอ"
ได้อะไร	ไฟล์ <code>.zip</code> สำหรับ import เข้า Power Automate
แก้ปัญหา	package ที่ import แล้วขึ้น connection ผิดบาทเท่า หรือ error <code>PackageFlowMissingConnectionMap</code>
ต้องเตรียม	ไฟล์ <code>.zip</code> ที่ export จาก tenant ของเราเป็นต้นแบบ ไม่มีแล้วทำไม่ได้จริง ๆ เพราะใน package มีรหัส connection และ GUID ที่ผูกกับ tenant ซึ่งเราไม่ได้และหาไม่ได้
สร้างต้นแบบ	Power Automate → Create → Automated cloud flow → trigger อะไรก็ได้ → 1 action → Save → My Flows → ... → Export → Package (.zip) · ใช้เวลา 2 นาที
ต้องรู้	legacy package จริงมี 5 ไฟล์ ไม่ใช่ 2 และสองไฟล์ในนั้น (<code>apisMap.json</code> , <code>connectionsMap.json</code>) ไม่ปรากฏในเอกสารที่ค้นเจอที่ไหนเลย · ไฟล์ export ผิง email ของคนที่ export ไว้ในเนื้อไฟล์ ระวังก่อนส่งออกนอกองค์กร

4.5 writing-app-manuals

พูดว่า	"เขียนคู่มือให้หน่อย"
ได้อะไร	คู่มือ admin และคู่มือผู้ใช้ เป็นไฟล์ <code>.docx</code> สองไฟล์ จาก spec เดียวกัน
แก้ปัญหา	คู่มือที่ไม่มีใครได้เขียนเพราะไม่มีเวลา และคู่มือสองเล่มที่เขียนแยกกันแล้วเนื้อหาเพี้ยนกันภายในสัปดาห์เดียว
ต้องรู้	มีโหมด draft ที่ใช้ก่อนสร้างระบบจริง เอาไปให้ user อ่าน ถ้า user บอกว่า "เราไม่ได้ทำแบบนี้" แปลว่าเพิ่งจับ requirement ที่ผิดได้ ในราคาหนึ่งประโยค · การถามว่า "approve spec นี้ได้ไหม" มักได้แค่การพยักหน้า

4.6 สรุปว่าตัวไหนรับอะไร ให้อะไร

skill	อ่านอะไร	สร้างอะไร
planning-powerplatform-solutions	การสัมภาษณ์	solution-spec.yaml, mockup, PDF
generating-powerapps-yaml	screens, variables, bindings	*.pa.yaml
building-sharepoint-lists	lists, relations	*.xlsx + สดริปต์
building-powerautomate-flows	flows	*.zip
writing-app-manuals	screens, roles, glossary	*.docx × 2

5 กติกาการใช้ให้ประหยัด token

4 ข้อ สำคัญพอ ๆ กับความถูกต้องเมื่อใช้ Pro plan

5.1 หนึ่ง stage ต่อหนึ่งแชท

เปิดแชทใหม่ทุกครั้งที่เปลี่ยนไปใช้ skill ตัวอื่น

เหตุผล: ถ้าวารรวมการสัมภาษณ์ requirement กับการสร้างของสามอย่างไว้ในแชทเดียว context จะถูกใช้ไปกับเรื่องที่ stage หลังไม่ต้องการ และคุณภาพจะตกก่อนที่ token จะหมด

5.2 แบน solution-spec.yaml เป็นไฟล์ ไม่ใช่ paste

แนบไฟล์แล้วพิมพ์ว่า "สร้าง SharePoint list ตาม spec นี้" เท่านั้น ไม่ต้องเล่าระบบใหม่ ไม่ต้อง paste YAML ทั้งไฟล์ลงในช่องแชท

5.3 prompt สั้นได้ เพราะกฎอยู่ใน skill แล้ว

ไม่ต้องเขียน prompt ยาวอธิบายว่าอยากได้ YAML แบบไหน ไม่ต้องแปะ convention ไม่ต้องเตือนเรื่อง property ordering ทั้งหมดนั้นอยู่ใน skill แล้ว การเขียนซ้ำคือการจ่าย token ซ้ำ

5.4 รวบรวมสิ่งที่จะแก้ไขให้ครบก่อนส่งกลับ

สะสมสิ่งที่ต้องแก้ไขให้ครบแล้วส่งทีเดียว ประหยัดกว่าส่งทีละข้อสามรอบ และผลลัพธ์มักดีกว่าเพราะมันเห็นภาพรวมของสิ่งที่ต้องการ

ถ้า **validator** เตือนขึ้นมาหลายสิบบรรทัดเพราะโปรเจกต์นี้ใช้สีและขนาดคนละชุด

อย่าแก้ skill และอย่าเริ่มมองข้าม warning ให้ใช้ prompt ในไฟล์ `prompts/project-style-override.md` วางตอนเริ่มแชท มันจะตั้งค่าสไตล์เฉพาะของโปรเจกต์นั้นให้ โดยไม่ต้องแก้ skill

อยากรู้ว่าที่เตือนมามีข้อไหนจริงจัง พิมพ์ว่า "validate with --platform-only แล้วเทียบสองรอบให้ดู" สิ่งที่ยหายไปคือเรื่องสไตล์ สิ่งที่ยังอยู่คือสิ่งที่ Studio จะปฏิเสธจริง

6 ข้อจำกัดของ validator

"ผ่าน validator" แปลว่า paste ได้
ไม่ได้แปลว่าหน้าจอออกมาถูก

6.1 ตรวจสอบอะไร ไม่ตรวจสอบอะไร

ตรวจให้

- control type และเวอร์ชันมีอยู่จริงหรือไม่
- property เรียงถูกต้องหรือไม่
- ค่าที่ต้องใส่ block scalar
- `X / Y` ที่ไม่ควรมี
- property ที่หน้าจอต้องมี
- enum member ที่ยังไม่มีใครยืนยัน

ตรวจไม่ได้

- กล่องสูงพอใส่ของข้างในหรือไม่
- เส้นขอบโดนขอบของกล่องแม่บังหรือไม่
- `=Parent.Width` ล้นกล่องแม่ที่มี padding
- อะไรก็ตามที่ขึ้นกับข้อมูลจริงตอน run

เหตุผล: validator อ่านตัวหนังสือ มันไม่ได้คำนวณ layout

6.2 กรณีจริงที่เคยเกิด

เวอร์ชันแรกของ validator ขึ้นข้อความว่า "Clean. Safe to paste into Power Apps Studio." ซึ่งเคลมเกินจริง เพราะสิ่งที่พิสูจน์ได้จริงมีแค่ไฟล์ parse ผ่าน

ผลคือมีคนรัน validator ผ่าน แล้วเชื่อผลนั้นสามารถติด ทั้งที่หน้าจอออกมาเพี้ยนทุกกรอบ

แก้แล้วในเวอร์ชัน **1.3.0** ตอนนี้ขึ้นว่า "No errors found. This should PARSE and paste." แล้วตามด้วยรายการสิ่งที่ไม่ได้ตรวจทุกครั้ง

6.3 วิธีทำงานที่เร็วที่สุด

- 1 รัน validator ให้ผ่านก่อน
- 2 paste เข้า Studio
- 3 ดูหน้าจอจริง
- 4 บอกมันว่าเห็นอะไรผิด เช่น "กล่องนี้เตี้ยไป ตัวหนังสือล้น"

การบอกสิ่งที่เห็นหนึ่งรอบ เร็วกว่าการเดาไปมาสามรอบ และเป็นรอบที่สายตาคนทำได้ดีกว่าเครื่องมือ

7 การดูแลและพัฒนา skill ต่อ

ทีมเป็นเจ้าของเองทั้งหมด ไม่ต้องใช้ GitHub ไม่ต้องเขียนโค้ด

7.1 ภาพรวมของวงจร

#	ขั้นตอน	ใครทำ	เมื่อไหร่
1	เก็บบันทึกท้ายแชท	ทุกคน	ทุกครั้งที่ทำงานจริงเสร็จ · 2 นาที
2	รอบปรับปรุง skill	ใครก็ได้ 1 คน	เมื่อสะสมบันทึกได้ 3-5 อัน · 30-45 นาที
3	แจกเวอร์ชันใหม่	ผู้ดูแลเวอร์ชัน	หลังทดสอบเวอร์ชันใหม่แล้ว

บทบาทที่ต้องมี

บทบาท	หน้าที่
ผู้ดูแลเวอร์ชัน 1 คน เท่านั้น	ดูแลโพลเดอร์กลาง เป็นคนเดียวที่วางไฟล์ลงโพลเดอร์ ปัจจุบัน/ และเป็นคนแจ้งทีมว่ามีเวอร์ชันใหม่ · จะสลับกันทำทุกไตรมาสก็ได้ แต่ต้องมีคนเดียวในแต่ละช่วงเวลา
ทุกคน	เก็บบันทึกหลังทำงานจริง · รับผิดชอบปรับปรุงได้ทุกคน

เหตุผลที่ต้องมีคนเดียว: ถ้าทุกคนวางไฟล์ได้ จะเกิดเวอร์ชันแยกกันหลายชุด และไม่มีใครรู้ว่าตัวไหนคือตัวจริง

7.1.1 โครงสร้างโพลเดอร์กลาง

สร้างใน SharePoint หรือ Teams ของทีม ตามนี้

Power Platform Skills/

- ─ ปัจจุบัน/ ← เวอร์ชันที่ทุกคนใช้อยู่
 - ─ generating-powerapps-yaml-v1.3.0.zip
 - ─ ... อีก 4 ไฟล์
- ─ เวอร์ชันเก่า/ ← เก็บย้อนหลังอย่างน้อย 3 เวอร์ชัน
- ─ บันทึก/ ← ไฟล์จากขั้นตอนที่ 1
 - ─ generating-powerapps-yaml/
 - ─ ใช้แล้ว/ ← ย้ายมาที่นี่หลังใช้ในรอบปรับปรุงแล้ว
- ─ prompts/ ← 3 ไฟล์ที่ได้รับมา
- ─ CHANGELOG.md ← ประวัติการแก้ไข ต่อท้ายทุกรอบ

ทำไมต้องเก็บเวอร์ชันเก่า: ถ้าเวอร์ชันใหม่ออกมาแย่กว่าเดิม ต้องถอยกลับได้ทันที การถอยกลับไม่ใช่ความล้มเหลว มันคือเหตุผลที่เก็บของเก่าไว้

7.2 ขั้นตอนที่ 1 — เก็บบันทึกท้ายแชท

ใครทำ	ทุกคน
เมื่อไหร่	ท้ายแชทที่ใช้ skill ทำงานจริงเสร็จ ก่อนปิดแชท
ใช้เวลา	2 นาที
ไฟล์	<code>prompts/team-quick-feedback.md</code>
ทำไมต้องท้ายแชท	แชทที่ยาวจะถูกลบความ และสิ่งที่เหลือรอดคือสรุปที่ฟังดูสมเหตุสมผล มากกว่าสรุปที่ถูกต้องตามวันศุกร์ถึงงานวันอังคาร จะได้คำตอบที่แต่งขึ้นโดยไม่รู้ตัว
อยากรองก่อนเก็บ	อย่าตัดสินใจเองว่าเรื่องนี้สำคัญพอหรือไม่ มีคนบันทึกไปแล้วหรือยัง การเก็บถูกมาก การตัดสินใจเป็นงานของขั้นตอนที่ 2 ซึ่งทำได้ดีกว่ามาก เมื่อมีบันทึกห้าอัน มากกว่ามีอันเดียว
เชฟยังง	ตั้งชื่อ <code>ปปป-ดต-ว-ชื่องาน.md</code> วางใน บันทึก/<ชื่อ skill>/ · ถ้ามีไฟล์ที่ใช้งานได้จริงรอบสุดท้าย ให้เก็บไว้คู่กันเสมอ บันทึกบอกว่าเกิดอะไรขึ้น แต่ไฟล์คือหลักฐาน

prompt ที่ต้องวาง — คัดลอกจากไฟล์ `team-quick-feedback.md` จะดีกว่าคัดลอกจาก PDF

```
We are done. Before I close this chat, write a short feedback record so the skill that
guided this work can be improved. Be blunt – this is for fixing the skill, not for
reassuring me.

Output one YAML code block and nothing else. No summary paragraph, and do not re-print
the artifact – I already have it.

meta:
  skill: <which skill guided this>
  skill_version: <the version comment in its SKILL.md, if you can see it>
  what_we_built: <one line>
  rounds: <how many times I sent it back for corrections>
  first_attempt_validated: <true|false – did your FIRST output pass the validator?>

# Exact text the platform showed. Copy it character for character, including the
# Location. Every verbatim error collected so far has become a static check.
studio_errors_verbatim: []

fixes:
  - element: <the exact control, column, or action. "the layout was wrong" is unusable>
    wrong: <what you produced>
    right: <what it had to become>
    caught_by: validator | studio | me-eyeballing | you-self-corrected
    tier: platform | house | unclear
    rule_status: absent | present-but-wrong | present-and-ignored
    rule_quote: <if present-but-wrong or present-and-ignored, quote the SKILL.md line>
    times: <rounds this same thing took>

biggest_miss: |
  <Free text, and answer it even if fixes is empty. What would have saved the most round
  trips if the skill had said it on page one?>

tier: `platform` = Studio or SharePoint REFUSED it. `house` = it worked, it just did not
match our conventions. Unsure? Write `unclear`.

rule_status: `absent` -> add a rule. `present-but-wrong` -> correct it.
`present-and-ignored` -> the rule was there, it was correct, and you did not follow it.
Say so. It means the rule is buried, or worded as advice where an instruction was needed.

Quote any value containing a colon-space: wrong: "Control: Image@2.2.0"
```

7.3 ขั้นตอนที่ 2 — รอบปรับปรุง skill

ใครทำ ใครก็ได้ 1 คน ไม่จำเป็นต้องเป็นผู้ดูแลเวอร์ชัน

เมื่อไหร่ เมื่อสะสมบันทึกได้ 3-5 อัน หรือทันทีเมื่อเจอเรื่องที่ Studio ปฏิเสธ

ใช้เวลา 30-45 นาที

ไฟล์ `prompts/improve-the-skill.md`

ต้องเตรียม

- 1 ไฟล์ `.zip` ของ skill เวอร์ชันล่าสุด จากโพลเดอร์ `ปัจจุบัน/`
- 2 บันทึกทั้งหมดที่สะสมไว้
- 3 แชนท์ใหม่ที่ว่างเปล่า อย่าใช้แชทเดิมที่ทำงาน บริบทเก่าจะทำให้ตัดสินใจเพี้ยน

ขั้นตอน

- 1 เปิดแชทใหม่ แนบไฟล์ `.zip` และบันทึกทั้งหมด
- 2 วาง prompt จากไฟล์ `improve-the-skill.md` ทั้งไฟล์
- 3 Claude จะทำตาม 7 ขั้นตอนของมันเอง ตอบคำถามที่มันถามระหว่างทาง
- 4 ตรวจสอบผลลัพธ์ — ต้องมีครบ 4 อย่าง: ไฟล์ `.zip` ใหม่, ตารางบอกว่าแก้ไฟล์ไหนไปบ้าง, ผลของ `self_check.py` ก่อนและหลัง, และบรรทัด `CHANGELOG`
- 5 โหลด `.zip` ใหม่ อัปโหลดเข้า `claude.ai` แล้วลองสร้างงานจริง **1 ชิ้น**
- 6 ถ้าผลดี ส่งให้ผู้ดูแลเวอร์ชันวางลงโพลเดอร์ `ปัจจุบัน/` · ถ้าแยกลง ทั้งแล้วใช้ของเดิม
- 7 ย้ายบันทึกที่ใช้ไปแล้วไปโพลเดอร์ `ใช้แล้ว/`

7.3.1 สิ่งที่ prompt นั้นบังคับให้ Claude ทำ

ไม่ต้องจำ แต่ควรรู้ไว้เพื่อตรวจสอบว่ามันทำครบหรือไม่

#	ขั้น	ทำอะไร
1	Baseline	รัน <code>self_check.py</code> จดตัวเลขไว้ ยังไม่ทำอะไร
2	จัดลำดับ	เรียงทุกเรื่องตามความสำคัญ เรื่องที่หลุดไปถึง Studio มาก่อนเสมอ
3	ตัดสินใจ	เลือกวิธีแก้ตาม <code>rule_status</code> ของแต่ละเรื่อง
4	เพิ่ม check ก่อนเพิ่มกฎ	check หุตุไฟล์ได้เอง กฎต้องอาศัยว่ามันอ่านและทำตาม
5	แก้ แล้วพิสูจน์	รัน <code>self_check.py</code> อีกครั้ง ต้องไม่มีอะไรพัง
6	เวอร์ชันและ CHANGELOG	รวมถึงต้องบอกด้วยว่าเรื่องไหนไม่ได้แก้ และเพราะอะไร
7	สร้าง zip ใหม่	พร้อมตารางว่าแก้ไฟล์ไหนไปบ้าง

`self_check.py` คืออะไร

สคริปต์ที่ติดมาในไฟล์ `.zip` ของ skill มันรัน validator กับหน้าจอ 17 หน้า ที่เคย compile ผ่าน Studio จริง แล้วเทียบกับตัวเลขที่บันทึกไว้

ถ้ากฎที่เพิ่งเพิ่มเข้าไป ทำให้หน้าจอ 17 หน้าขึ้น **warning** แปลว่ากฎนั้นผิด ไม่ใช่หน้าจอผิด เพราะหน้าจอเหล่านั้นใช้งานได้จริงมาแล้ว

เคยเกิดขึ้นแล้วหนึ่งครั้ง มีการเพิ่ม warning ที่ดูสมเหตุสมผลมาก แล้วมันไปเตือนปุ่ม 20 ปุ่มที่ใช้งานได้ปกติ ตัวเลขชนะการคาดเดา ทุกครั้ง

ถ้าผลลัพธ์ไม่ครบ 4 อย่าง

สั่งมันตรง ๆ ว่าขาดอะไร โดยเฉพาะสองอย่างนี้ ซึ่งเป็นสองอย่างที่ถูกข้าม

ผล <code>self_check</code>	ถ้ามันไม่ได้รัน ให้สั่งว่า "run scripts/self_check.py and paste the full output, before and after" · ถ้ามันบอกว่ารันไม่ได้ อย่ารีบงานรอบนั้น
เรื่องที่ไม่ได้แก้	ถ้ามันไม่ได้บอก ให้ถามว่า "which findings did you not act on, and why?" · คนที่ส่งบันทึกมาแล้วไม่เห็นอะไรเกิดขึ้น จะเลิกส่ง แล้วข้อมูลนั้นหายไปถาวร

7.4 กฎ 7 ข้อของการแก้ skill

กฎเหล่านี้อยู่ใน prompt แล้ว แต่คนที่รันรอบปรับปรุงควรรู้ด้วย เพราะเป็นคนที่ต้องตรวจว่า Claude ทำตามหรือไม่

#	กฎ	เหตุผล
1	ทุกกฎต้องมีหลักฐาน	ตัวเลขที่นับได้ หรือข้อความ error จากแพลตฟอร์มแบบคำต่อคำ คำว่า "น่าจะดี" หรือ "โดยทั่วไปแนะนำว่า" ไม่ใช่หลักฐาน
2	platform แปลว่าแพลตฟอร์มปฏิเสธจริง	ไม่ใช่ "ดูไม่สวย" ไม่ใช่ "ไม่สม่ำเสมอ" การเอาความชอบของทีมไปตัดสินว่าเป็นกฎของแพลตฟอร์ม คือสาเหตุที่เครื่องมือใช้กับโปรเจกต์ถัดไปไม่ได้
3	ห้ามแก้ validator เพื่อให้ warning หายไป	ถ้า warning ผิด แปลว่ากฎเบื้องหลังผิด หรือโปรเจกต์นี้ใช้ convention คนละชุด อย่างแรกแก้ที่กฎ อย่างหลังแก้ที่ <code>house-style.yaml</code>
4	SKILL.md ต้องไม่เกิน 200 บรรทัด	ความยาวมีต้นทุนที่ผู้อ่านทุกคนในอนาคตต้องจ่าย ถ้าหาอะไรตัดออกไม่ได้ สิ่งที่จะเพิ่มอาจไม่คุ้มที่
5	ห้ามปล่อยเวอร์ชันที่ <code>self_check.py</code> ไม่ผ่าน	ไม่มีข้อยกเว้น
6	ห้ามลบกฎที่มีตัวเลขกำกับ	เว้นแต่มีบันทึกจากงานจริงที่ขัดแย้งกับมัน มีคนนับตัวเลขนั้นมาแล้ว
7	ไม่แน่ใจให้บอกว่าไม่แน่ใจ	คำว่า "แยกไม่ออกว่าอันนี้เป็น platform หรือ house" มีค่ามากกว่าการเดาแล้วติดป้ายผิด ทั้งไว้รอบหน้า แล้วเขียนไว้ใน CHANGELOG

7.5 สิ่งที่ต้องทำหลังปล่อยเวอร์ชันใหม่

- วางไฟล์ใน `ปัจจุบัน/` ตั้งชื่อให้มีเลขเวอร์ชัน เช่น `generating-powerapps-yaml-v1.4.0.zip`
- ย้ายไฟล์เก่าไป `เวอร์ชันเก่า/`
- ต่อท้าย `CHANGELOG.md`
- แจ้งทีม พร้อมบอกชัดว่าต้องโหลดใหม่หรือไม่

ข้อ 4 คือข้อที่มักถูกลืม และการลืมทำให้งานทั้งรอบสูญเปล่า

Skill ไม่ sync ระหว่างคน เวอร์ชันใหม่ที่วางไว้ในโพลเดอร์แต่ไม่มีใครโหลด ก็เท่ากับไม่ได้ทำอะไรเลย รอบปรับปรุงจบเมื่อทีมใช้เวอร์ชันใหม่แล้ว ไม่ใช่เมื่อไฟล์ถูกวางเสร็จ

8 คำถามที่พบบ่อย

8.1 ต้องใช้ GitHub หรือลงโปรแกรมอะไรไหม

ไม่ต้อง ใช้แค่ claude.ai กับเบราร์เซออร์ไฟล์ทั้งหมดอยู่ในโพลเดอร์กลางของทีม

8.2 ลง skill แล้ว คนอื่นในทีมได้ด้วยไหม

ไม่ได้ Skill บน claude.ai เป็นของแต่ละ account และไม่ sync กัน ทุกคนต้องอัปโหลดเอง

8.3 ต้องลงครบทั้ง 5 อันไหม

ลงครบดีที่สุด เพราะถูกเขียนให้ส่งงานต่อกัน ถ้าจะเริ่มอันเดียวก่อน ให้เริ่มที่ `generating-powerapps-yaml`

8.4 จะกิน token เยอะไหม

น้อยกว่าเดิม เพราะถูกอยู่ใน skill แล้ว ไม่ต้องพิมพ์อธิบายซ้ำทุกแชท และไม่ต้องdingงานกลับไปมาหลายรอบ ซึ่งรอบที่ตีกลับคือส่วนที่แพงที่สุด ทำตามส่วนที่ 5 ให้ครบ

8.5 ถ้าแก้ skill แล้วมันแย่ง

กลับไปใช้เวอร์ชันก่อนหน้าจากโพลเดอร์ `เวอร์ชันเก่า/` แล้วบันทึกไว้ว่ารอบนั้นเปลี่ยนอะไรและอาการเป็นอย่างไร

8.6 ไม่มีใครในทีมเขียนโค้ดเป็น จะแก้ skill ได้จริงหรือ

ได้ prompt ในไฟล์ `improve-the-skill.md` ทำงานทั้งหมดให้ รวมถึงการรันสคริปต์ตรวจสอบและสร้างไฟล์ `.zip` ใหม่ หน้าที่ของคนคือตรวจว่าผลลัพธ์มีครบ 4 อย่างตามส่วนที่ 7.3 และตัดสินใจว่าจะรับหรือไม่

8.7 ถ้า Claude สร้างไฟล์ .zip ให้ไม่ได้

prompt สั่งให้มันบอกเองและส่งไฟล์ที่แก้แล้วออกมาเป็นข้อความแทน วิธีประกอบเอง: แยกไฟล์ zip เดิม เอาไฟล์ใหม่ทับไฟล์เดิม แล้วบีบอัดโพลเดอร์นั้นใหม่ (Windows: คลิกขวาที่โพลเดอร์ → Send to → Compressed folder) ชื่อโพลเดอร์ด้านในต้องเป็นชื่อ skill และต้องมี `SKILL.md` อยู่ในนั้น

8.8 skill ผ่านการใช้งานจริงแล้วหรือยัง

ผ่านแล้วหนึ่งโปรเจกต์ที่ไม่เกี่ยวข้องกับโปรเจกต์ต้นทางเลย ได้ผลลัพธ์ 191 controls โดยไม่มี control type หรือ property ที่ไม่มีอยู่จริงแม้แต่ตัวเดียว ส่วนที่พลาดกลายเป็นเนื้อหาในส่วนที่ 6

8.9 เอกสารนี้แก้ไขได้ไหม

ได้ไฟล์ต้นฉบับคือ `handoff/handoff.html` แก้ข้อความแล้วสร้าง PDF ใหม่ด้วย `python3 scripts/build_handoff.py`

9 Checklist หน้าเดียว

9.1 ครั้งแรกที่เริ่มใช้

- ☐ อัปโหลด skill ทั้ง 5 อันที่ Settings → Capabilities → Skills
- ☐ สร้างโพลเดอร์กกลางตามโครงสร้างในส่วนที่ 7.1.1
- ☐ ตกลงกันว่าใครเป็นผู้ดูแลเวอร์ชัน
- ☐ วางไฟล์ prompt ทั้ง 3 ไฟล์ลงโพลเดอร์กกลาง

9.2 ทุกโปรเจกต์

- ☐ แชนท์ที่ 1 วางระบบ ได้ solution-spec.yaml + mockup + PDF
- ☐ เอา PDF ไปคุยกับ user ก่อนสร้างอะไร
- ☐ แชนท์ที่ 2-4 สร้าง SharePoint, Power Apps, flow — แนบ spec ทุกครั้ง
- ☐ paste เข้า Studio แล้วดูหน้าจอลงก่อนเชื่อว่าเสร็จ
- ☐ แชนท์ที่ 5 เขียนคู่มือ

9.3 ทุกครั้งก่อนปิดแชททำงาน

- ☐ วาง prompt จาก team-quick-feedback.md
- ☐ เซฟบันทึกลง บันทึก/<ชื่อ skill>/ ตั้งชื่อเป็นวันที่
- ☐ เก็บไฟล์ที่ใช้งานได้จริงไว้คู่กัน ถ้ามี

9.4 รอบปรับปรุง — เมื่อมีบันทึก 3-5 อัน

- ☐ เปิดแชทใหม่ แนบ .zip และบันทึกทั้งหมด
- ☐ วาง prompt จาก improve-the-skill.md
- ☐ ตรวจสอบว่าได้ครบ 4 อย่าง — zip ใหม่ · ตารางไฟล์ที่แก้ · ผล self_check.py · บรรทัด CHANGELOG
- ☐ อัปโหลดแล้วลองสร้างงานจริง 1 ชิ้นก่อนแจก
- ☐ ผู้ดูแลเวอร์ชันวางไฟล์ลง ปัจจุบัน/ · ย้ายของเก่าไป เวอร์ชันเก่า/
- ☐ ต่อท้าย CHANGELOG.md
- ☐ แจ้งทีมว่าต้องโหลดใหม่หรือไม่
- ☐ ย้ายบันทึกที่ใช้แล้วไป ใช้แล้ว/

ข้อเดียวที่สำคัญที่สุดคือ 9.3 — วาง prompt เก็บบันทึกท้ายแชท ใช้เวลา 2 นาที

ถ้าไม่มีใครเก็บบันทึก จะไม่มีอะไรให้ปรับปรุง และ skill เหล่านี้จะค่อย ๆ เก่าลงจนไม่ตรงกับงานจริง